

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Engenharia de Software - Laboratório de Experimentação de Software -
Um estudo das características de qualidade de sistemas java

Alunos: Ana Luiza Santos Gomes - Bruna Barbosa Portilho Bernardes -
Walter Roberto Rodrigues Louback

Belo Horizonte, Abril/2026

Introdução

Com o crescimento do desenvolvimento open source, plataformas como o GitHub tornaram-se ambientes fundamentais para colaboração e evolução de software. Nesse cenário, entender como características desses projetos se relacionam com a qualidade do código torna-se essencial tanto para desenvolvedores quanto para pesquisadores da área de Engenharia de Software.

Além disso, a qualidade de software é um aspecto fundamental no desenvolvimento de sistemas, impactando diretamente na manutenibilidade, confiabilidade e evolução dos projetos. Nesse contexto, métricas de código-fonte são amplamente utilizadas para avaliar atributos internos do software, como acoplamento, coesão e complexidade.

Este trabalho tem como objetivo analisar a qualidade de repositórios Java disponíveis no GitHub, utilizando métricas extraídas pela ferramenta CK. Além disso, busca-se investigar a relação entre características dos repositórios como popularidade, maturidade e tamanho e a qualidade interna do código.

A análise desses fatores permite identificar possíveis padrões e tendências, contribuindo para uma melhor compreensão de como atributos externos de um projeto influenciam sua estrutura interna.

Apesar da ampla disponibilidade de repositórios open source no GitHub, ainda não está claro como características externas dos projetos como popularidade, maturidade, atividade e tamanho influenciam a qualidade interna do código. Dessa forma, surge a necessidade de investigar se tais atributos podem ser utilizados como indicadores indiretos de qualidade de software.

2. Fundamentação Teórica

Métricas de software são amplamente utilizadas para avaliar atributos internos de sistemas, como complexidade, acoplamento e coesão. Entre as métricas mais comuns, destacam-se o CBO (Coupling Between Objects), que mede o grau de dependência entre classes, o DIT (Depth of Inheritance Tree), que indica a profundidade da hierarquia de herança, e o LCOM (Lack of Cohesion of Methods), que avalia o nível de coesão interna de uma classe.

A ferramenta CK é amplamente utilizada para a extração dessas métricas em projetos orientados a objetos, permitindo a análise automatizada de código-fonte Java. Estudos recentes têm utilizado essas métricas para investigar a relação entre qualidade de software e características de projetos, como popularidade e tamanho.

3. Metodologia

A metodologia adotada baseia-se na coleta automatizada e análise de métricas de software em repositórios Java.

Inicialmente, foi utilizada uma base contendo os principais repositórios Java do GitHub. A partir dessa base, foi desenvolvido um script em Python responsável por:

- Clonar automaticamente os repositórios;
- Identificar a presença de arquivos `.java`;
- Executar a ferramenta CK para extração de métricas;
- Consolidar os resultados em arquivos CSV.

A ferramenta CK foi utilizada para extrair métricas estruturais de código, incluindo:

- **CBO (Coupling Between Objects):** mede o acoplamento entre classes;
- **DIT (Depth of Inheritance Tree):** mede a profundidade da hierarquia de herança;
- **LCOM (Lack of Cohesion of Methods):** mede a falta de coesão das classes;
- **LOC (Lines of Code):** quantidade de linhas de código.

Para cada repositório, os dados foram sumarizados por meio de medidas estatísticas, como média, mediana e desvio padrão.

O experimento foi conduzido seguindo as etapas descritas abaixo:

- Seleção dos repositórios a partir da base de dados;
- Clonagem automática dos repositórios utilizando scripts em Python;
- Verificação da presença de arquivos Java;
- Execução da ferramenta CK para extração das métricas;
- Coleta dos arquivos gerados (class.csv e method.csv);
- Consolidação dos dados em um arquivo CSV final;
- Análise estatística das métricas coletadas.

Métrica	Descrição	Interpretação
CBO	Acoplamento entre classes	Valores menores indicam menor dependência
LCOM	Falta de coesão	Valores menores indicam menor dependência
DIT	Profundidade da árvore de herança	Valores menores indicam melhor coesão

LOC	Linhas de código (tamanho do sistema)	Indica o tamanho do sistema
-----	---------------------------------------	-----------------------------

Durante a execução do experimento, foram adotadas algumas decisões importantes para garantir a qualidade dos dados:

- Repositórios sem arquivos `.java` foram descartados;
- Foi considerado um número mínimo de arquivos Java para análise;
- Repositórios com erro de clonagem foram ignorados;
- Foi utilizada uma amostra de repositórios devido a limitações computacionais.

A análise dos dados foi realizada por meio de gráficos de dispersão, permitindo observar possíveis correlações entre as variáveis estudadas.

4. Objetivos e Hipóteses

O objetivo deste trabalho é analisar a qualidade de repositórios Java por meio de métricas estruturais extraídas automaticamente, investigando a relação entre atributos externos dos projetos e características internas do código, a fim de identificar padrões e possíveis indicadores de qualidade em projetos open source.

Para isso, foram definidas as seguintes questões de pesquisa e com base nessas questões, foram formuladas as seguintes hipóteses:

- **RQ 01.** Qual a relação entre a popularidade dos repositórios e as suas características de qualidade?

Métrica de Popularidade: Número de estrelas

Métricas de Qualidade: CBO (Coupling Between Objects), LCOM (Lack of Cohesion of Methods), DIT (Depth of Inheritance Tree)

Hipótese Informal: Repositórios mais populares (com maior número de estrelas) tendem a apresentar melhores características de qualidade (CBO mais baixo, DIT mais alto, LCOM mais baixo). Isso ocorre porque projetos populares atraem mais colaboradores e estão sujeitos a um escrutínio maior, incentivando a adoção de boas práticas de design e codificação.

- **RQ 02.** Qual a relação entre a maturidade do repositórios e as suas características de qualidade?

Métrica de Maturidade: Idade (em anos) do repositório

Métricas de Qualidade: CBO (Coupling Between Objects), LCOM (Lack of Cohesion of Methods), WMC (Weighted Methods per Class)

Hipótese Informal: Repositórios mais maduros (com maior idade) tendem a apresentar melhores características de qualidade até um certo ponto. No início, a qualidade pode aumentar com o tempo devido a refatorações e correções de bugs. No entanto, após um longo período, a qualidade pode começar a declinar devido à acumulação de dívida técnica e à dificuldade de adaptar o código a novas tecnologias.

- **RQ 03.** Qual a relação entre a atividade dos repositórios e as suas características de qualidade?

Métrica de Atividade: Número de releases

Métricas de Qualidade: CBO (Coupling Between Objects), LCOM (Lack of Cohesion of Methods), Número de Commits

Hipótese Informal: Repositórios com maior atividade (maior número de releases) tendem a apresentar melhores características de qualidade. A atividade constante indica que o projeto está sendo mantido ativamente, com refatorações, melhorias e correções de bugs, o que contribui para a qualidade do código.

- **RQ 04.** Qual a relação entre o tamanho dos repositórios e as suas características de qualidade?

Métrica de Tamanho: Linhas de código (LOC)

Métricas de Qualidade: CBO (Coupling Between Objects), LCOM (Lack of Cohesion of Methods), NOC (Number of Children)

Hipótese Informal: Repositórios maiores (com mais linhas de código) tendem a apresentar piores características de qualidade. Projetos grandes podem se tornar mais complexos e difíceis de manter, levando a um aumento no acoplamento (CBO), menor coesão (LCOM) e hierarquias de herança mais complexas (NOC). No entanto, essa relação pode ser atenuada se o projeto for bem modularizado e seguir boas práticas de design.

5. Execução do Experimento

O experimento foi executado de forma automatizada por meio de um script que percorre a lista de repositórios e realiza o processamento individual de cada um.

Para cada repositório, o fluxo de execução seguiu as etapas:

1. Clonagem do repositório;
2. Verificação da existência de arquivos Java;
3. Execução da ferramenta CK;
4. Coleta dos arquivos gerados;
5. Consolidação dos dados em um arquivo final.

Durante a execução, foram identificados alguns problemas:

- Repositórios sem arquivos Java, sendo automaticamente descartados;
- Erros relacionados ao limite de tamanho de caminho no sistema operacional Windows;
- Repositórios muito grandes, com tempo elevado de processamento.

Devido a essas limitações, foi utilizada uma amostra de repositórios para validação do experimento, garantindo a viabilidade da execução e a qualidade dos dados coletados.

6. Resultados

Os resultados obtidos foram armazenados em um arquivo CSV contendo métricas por repositório.

Para cada projeto analisado, foram coletadas informações como:

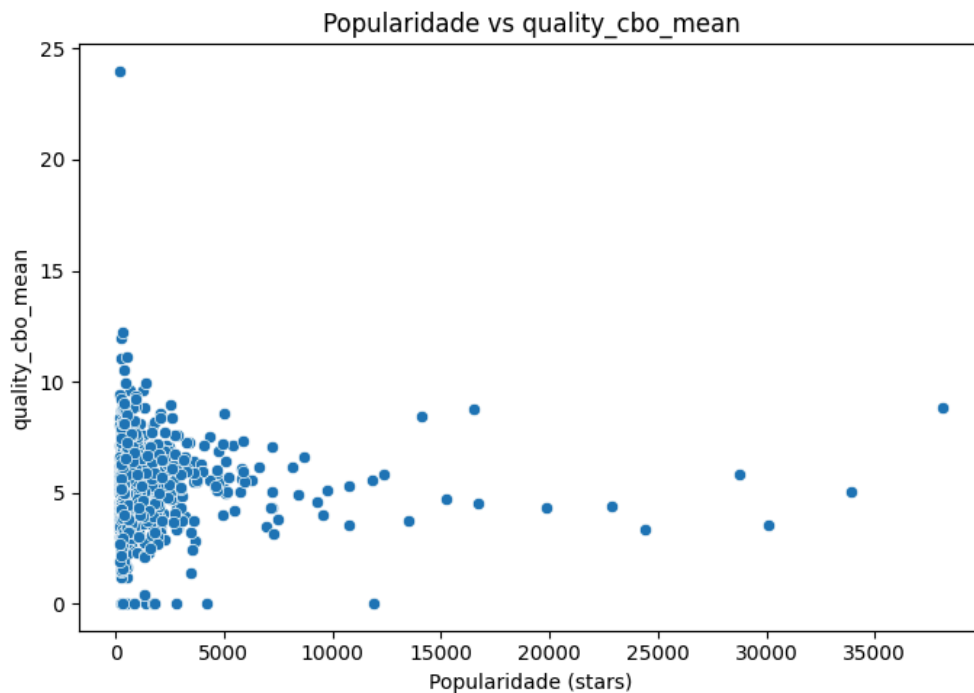
- Número de arquivos Java;
- Quantidade de classes;
- Métricas CK (CBO, DIT, LCOM, LOC);
- Medidas estatísticas (média, mediana e desvio padrão).

RQ 01. Qual a relação entre a popularidade dos repositórios e as suas características de qualidade?

Métrica de Popularidade: Número de estrelas

Métricas de Qualidade: CBO (Coupling Between Objects), LCOM (Lack of Cohesion of Methods), DIT (Depth of Inheritance Tree)

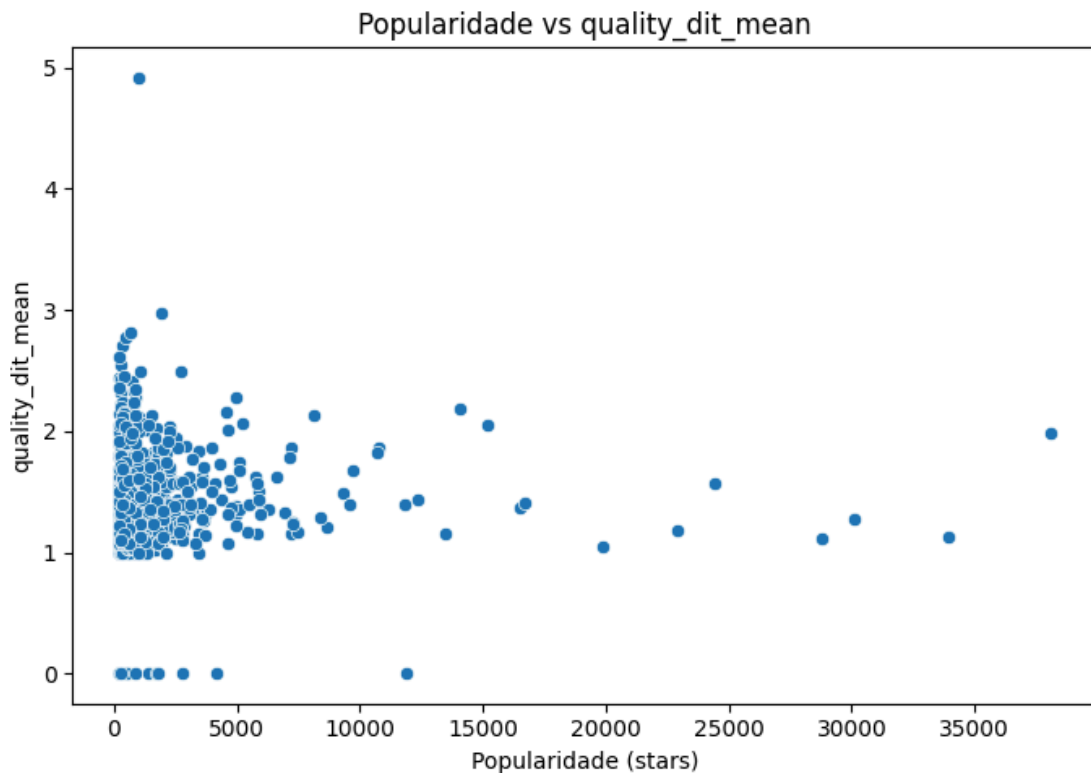
- Figura 1 – Relação entre popularidade (stars) vs CBO:



O gráfico apresenta a relação entre a popularidade dos repositórios, medida pelo número de estrelas, e o acoplamento entre classes (CBO). Observa-se uma grande dispersão dos dados, sem a formação de um padrão linear evidente. A maioria dos repositórios, independentemente do número de estrelas, apresenta valores moderados de CBO.

Isso indica que a popularidade de um projeto não está diretamente associada a menores níveis de acoplamento. Embora alguns repositórios mais populares apresentem valores mais consistentes, não é possível afirmar que projetos com mais estrelas possuem melhor qualidade estrutural.

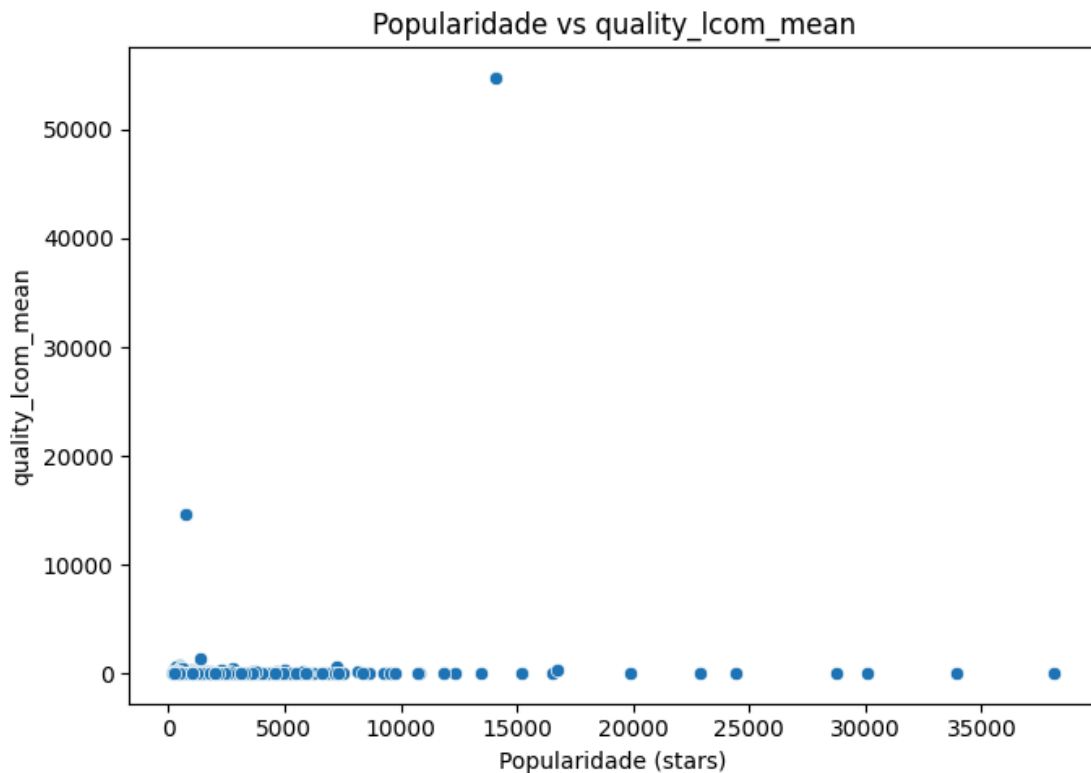
- Figura 2 – Relação entre popularidade (stars) vs DIT:



O gráfico apresenta a relação entre a popularidade dos repositórios, medida pelo número de estrelas, e a profundidade da árvore de herança (DIT). Observa-se uma concentração significativa de pontos em baixos valores de estrelas e DIT, indicando que a maioria dos projetos possui estruturas de herança pouco profundas.

Não há evidência de uma tendência clara de aumento do DIT com o crescimento da popularidade. Repositórios mais populares aparecem de forma dispersa, sem indicar aumento consistente na complexidade de herança. Isso sugere que a popularidade não influencia diretamente o uso de hierarquias mais profundas no código.

- Figura 3 – Relação entre popularidade (stars) vs LCOM:



O gráfico relaciona a popularidade dos repositórios com a métrica LCOM, que indica a falta de coesão das classes. Observa-se uma distribuição bastante dispersa, com valores variados de LCOM em diferentes níveis de popularidade.

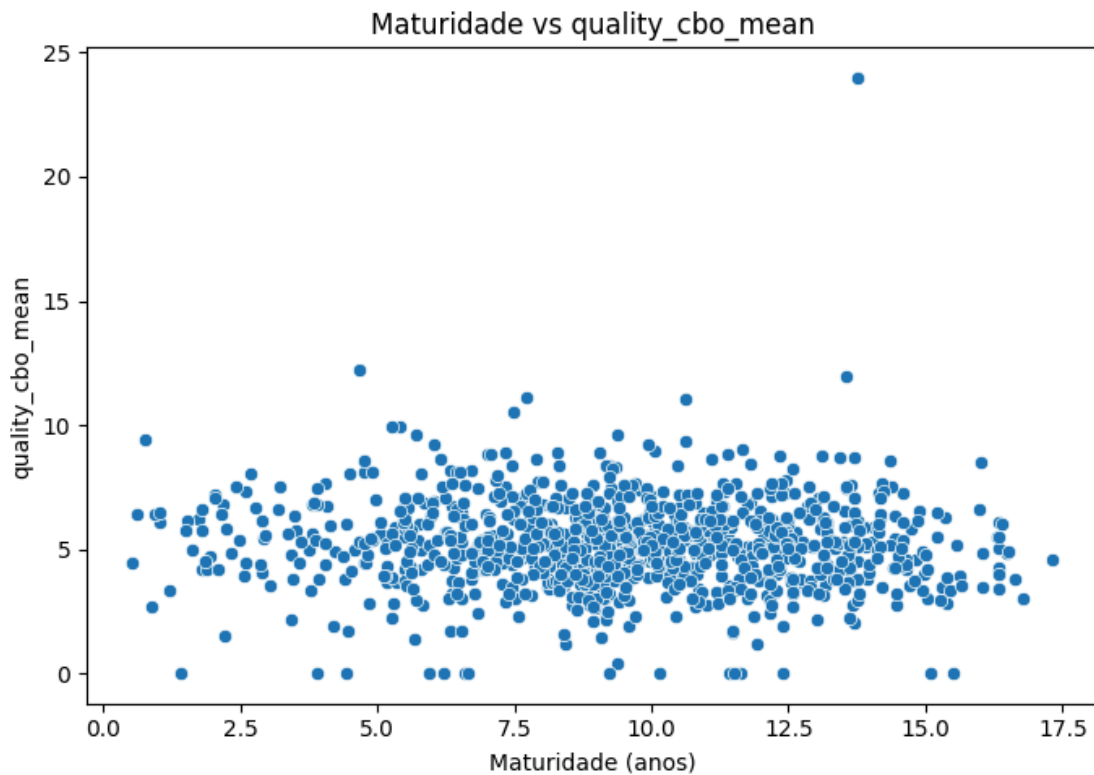
Esse comportamento sugere que a popularidade não influencia diretamente a coesão interna do código. Existem repositórios populares com boa coesão, mas também há casos com baixa coesão, indicando ausência de uma relação consistente.

RQ 02. Qual a relação entre a maturidade do repositórios e as suas características de qualidade?

Métrica de Maturidade: Idade (em anos) do repositório

Métricas de Qualidade: CBO (Coupling Between Objects), LCOM (Lack of Cohesion of Methods), WMC (Weighted Methods per Class)

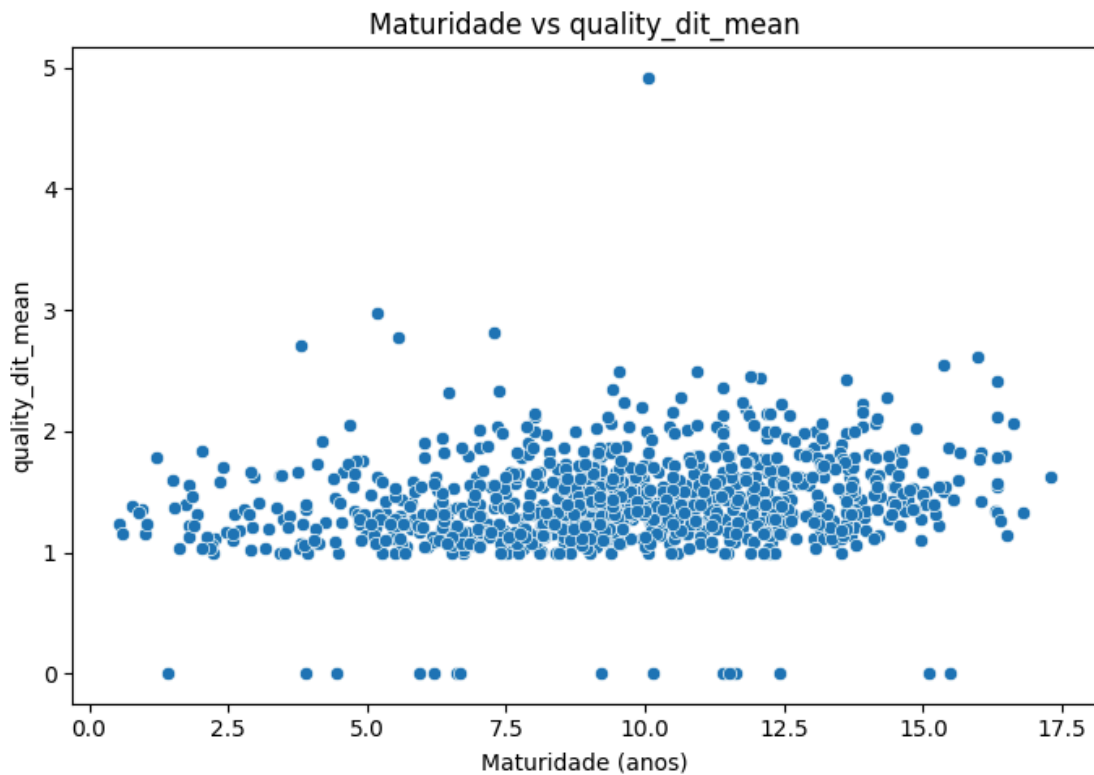
- Figura 4 – Relação entre maturidade (anos) vs CBO:



O gráfico apresenta a relação entre a idade dos repositórios e o acoplamento entre classes. Observa-se uma grande dispersão dos dados, com repositórios antigos e recentes apresentando valores variados de CBO.

Isso indica que a maturidade do sistema não é um fator determinante para o nível de acoplamento, sugerindo que boas práticas de desenvolvimento podem ser adotadas independentemente do tempo de existência do projeto.

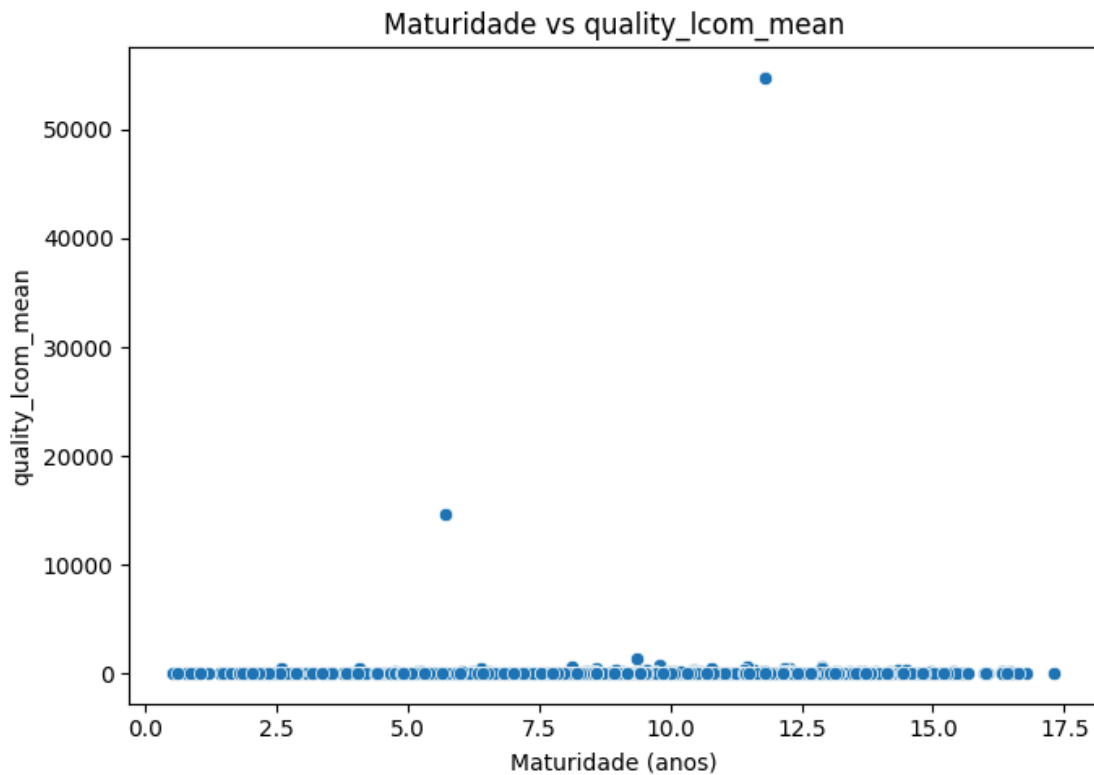
- Figura 5 – Relação entre maturidade (anos) vs DIT:



Neste gráfico, é analisada a relação entre a idade dos repositórios e a profundidade da árvore de herança (DIT). Observa-se uma leve tendência de aumento nos valores de DIT conforme a idade do repositório cresce.

Esse comportamento sugere que sistemas mais antigos tendem a apresentar estruturas mais complexas de herança, possivelmente devido à evolução contínua e à incorporação de novas funcionalidades ao longo do tempo.

- Figura 6 – Relação entre maturidade (anos) vs LCOM:



O gráfico relaciona a idade dos repositórios com a métrica LCOM, que indica a falta de coesão das classes. Observa-se uma distribuição dispersa dos dados, com repositórios antigos e recentes apresentando diferentes níveis de coesão.

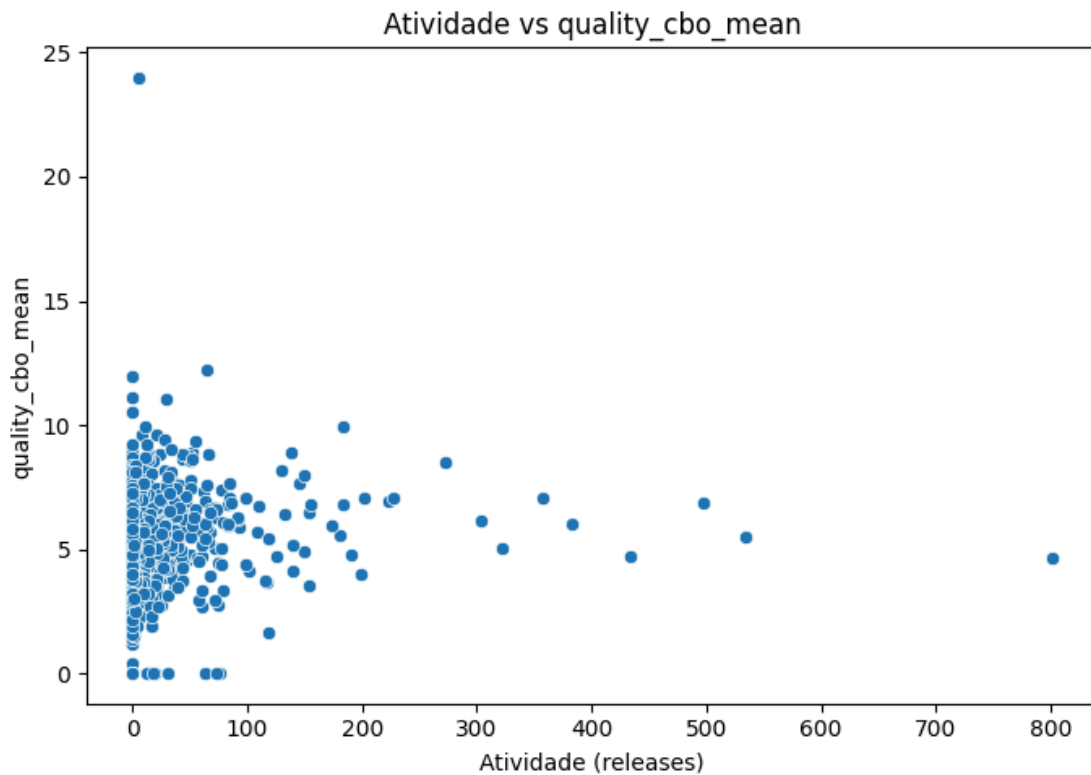
Não é possível identificar uma tendência clara de melhora ou piora da coesão com o aumento da maturidade. Esse comportamento indica que o tempo de existência do projeto não é um fator determinante para a qualidade da coesão interna das classes.

RQ 03. Qual a relação entre a atividade dos repositórios e as suas características de qualidade?

Métrica de Atividade: Número de releases

Métricas de Qualidade: CBO (Coupling Between Objects), LCOM (Lack of Cohesion of Methods), Número de Commits

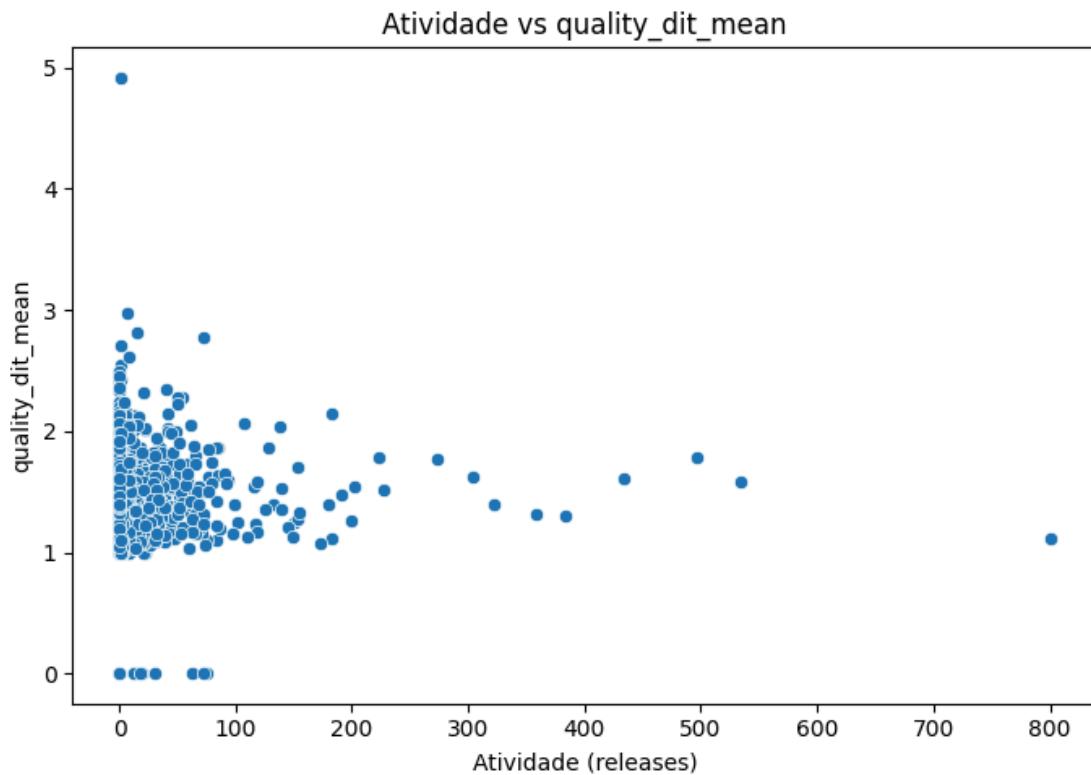
- Figura 7 – Relação entre atividade (releases) vs CBO:



Este gráfico mostra a relação entre a quantidade de releases de um repositório e o nível de acoplamento entre classes. Os dados apresentam alta dispersão, sem tendência clara de crescimento ou redução.

Isso sugere que a frequência de releases não está diretamente relacionada à qualidade estrutural do código, podendo refletir apenas o ritmo de desenvolvimento e entrega de versões.

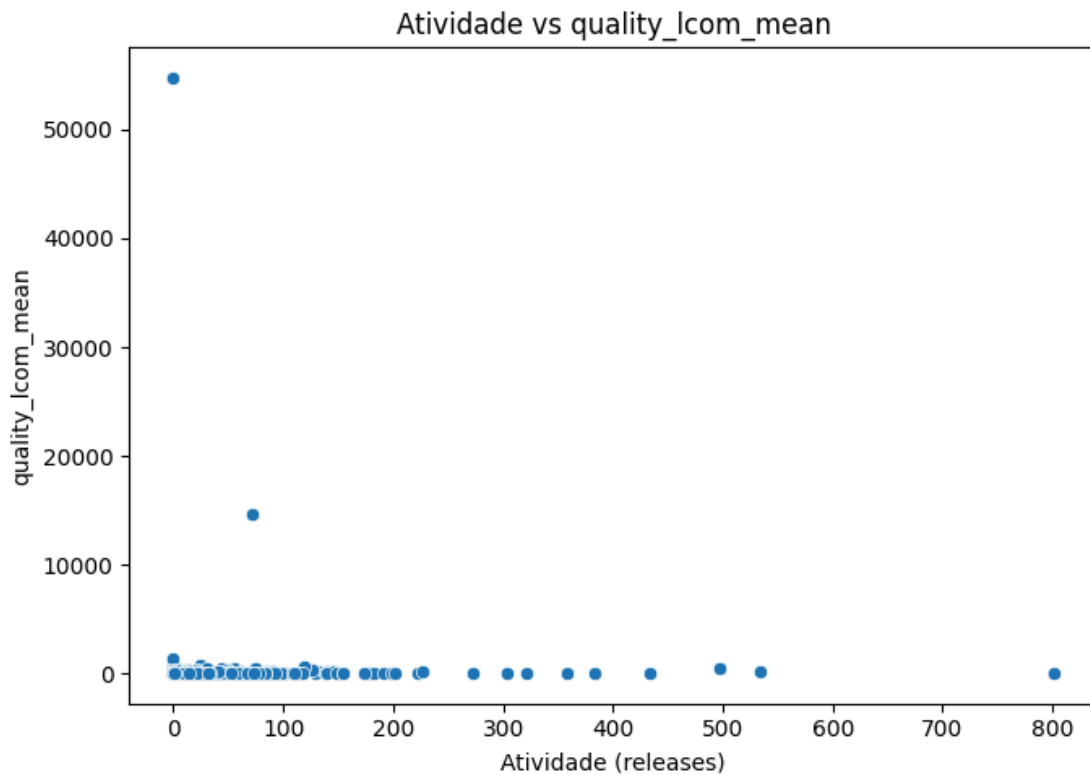
- Figura 8 – Relação entre atividade (releases) vs DIT:



O gráfico apresenta a relação entre a quantidade de releases e a profundidade da árvore de herança (DIT). Observa-se que a maioria dos repositórios possui baixo número de releases e valores reduzidos de DIT.

A distribuição dos pontos não indica qualquer tendência clara entre as variáveis. Mesmo repositórios com maior número de releases não apresentam aumento significativo na profundidade da herança, sugerindo que a atividade do projeto não impacta diretamente essa métrica estrutural.

- Figura 9 – Relação entre atividade (releases) vs LCOM:



O gráfico relaciona o número de releases com a métrica LCOM. Observa-se novamente uma distribuição dispersa dos pontos, indicando ausência de padrão significativo.

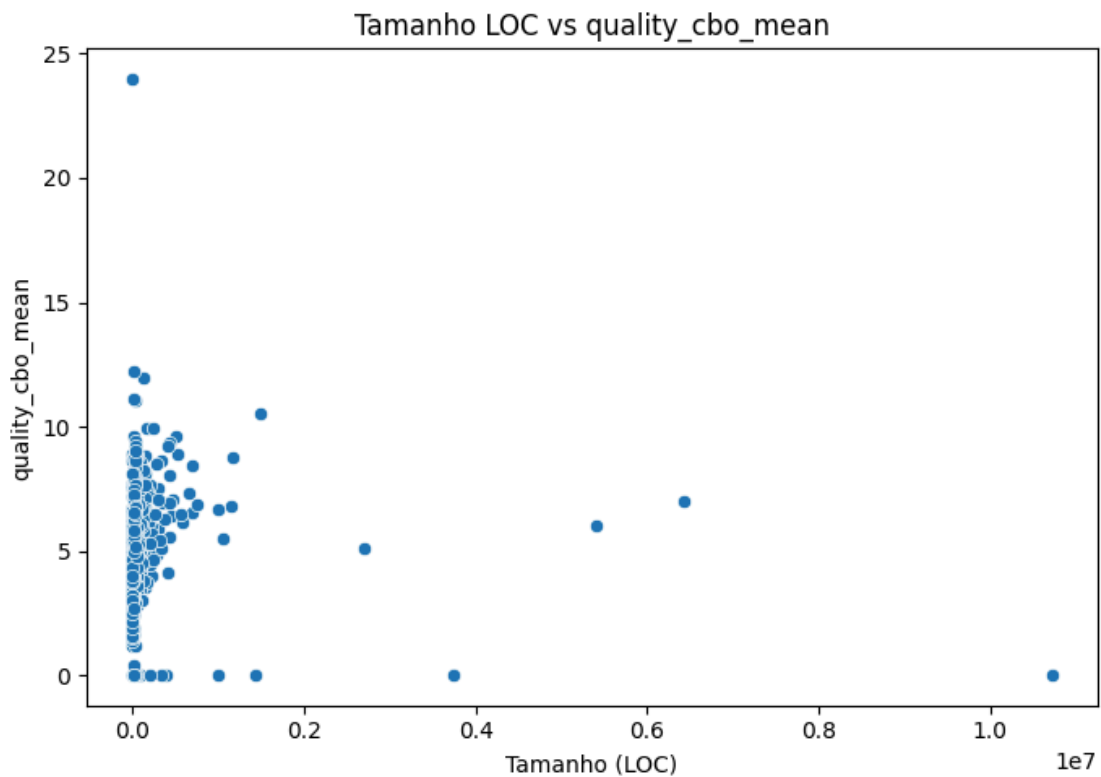
Esse resultado indica que a atividade do projeto, medida pelo número de releases, não garante maior coesão do código, reforçando que qualidade estrutural depende de fatores adicionais.

RQ 04. Qual a relação entre o tamanho dos repositórios e as suas características de qualidade?

Métrica de Tamanho: Linhas de código (LOC)

Métricas de Qualidade: CBO (Coupling Between Objects), LCOM (Lack of Cohesion of Methods), NOC (Number of Children)

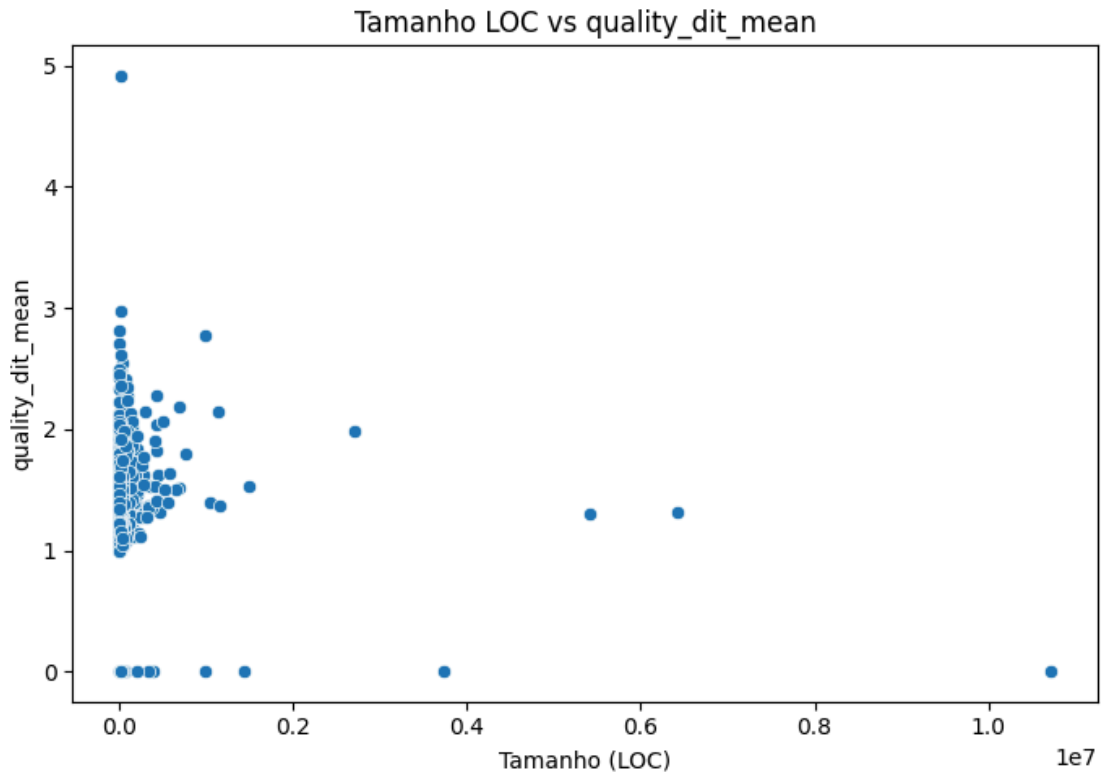
- Figura 10 – Relação entre tamanho (LOC) vs CBO:



O gráfico apresenta a relação entre o tamanho dos repositórios, medido por linhas de código (LOC), e o acoplamento entre classes (CBO). Observa-se uma tendência de aumento na variabilidade do CBO conforme o tamanho do sistema cresce.

Isso indica que sistemas maiores tendem a apresentar maior complexidade estrutural e níveis mais elevados de acoplamento, tornando sua manutenção mais desafiadora.

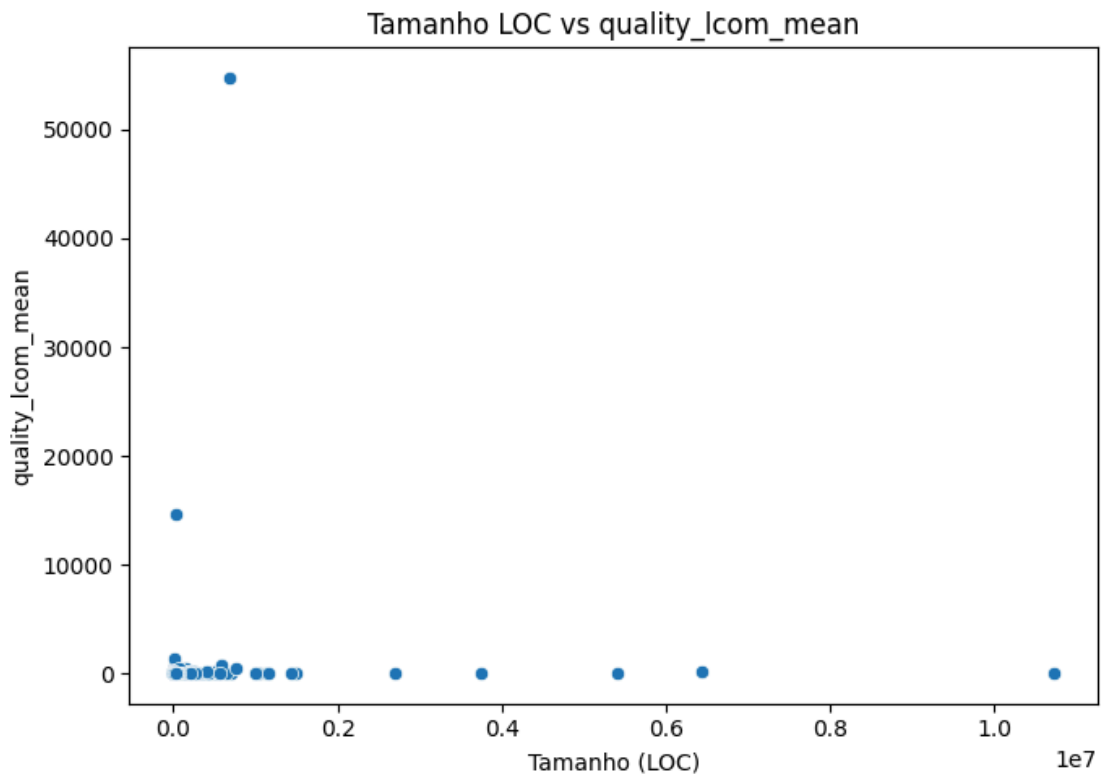
- Figura 11 – Relação entre tamanho (LOC) vs DIT:



O gráfico analisa a relação entre o tamanho dos repositórios, medido por linhas de código (LOC), e a profundidade da árvore de herança (DIT). Observa-se que a maioria dos sistemas, independentemente do tamanho, apresenta valores relativamente baixos de DIT.

Apesar da presença de alguns pontos isolados com valores mais elevados, não há uma tendência clara de aumento do DIT com o crescimento do tamanho do sistema. Isso sugere que sistemas maiores não necessariamente utilizam estruturas de herança mais profundas.

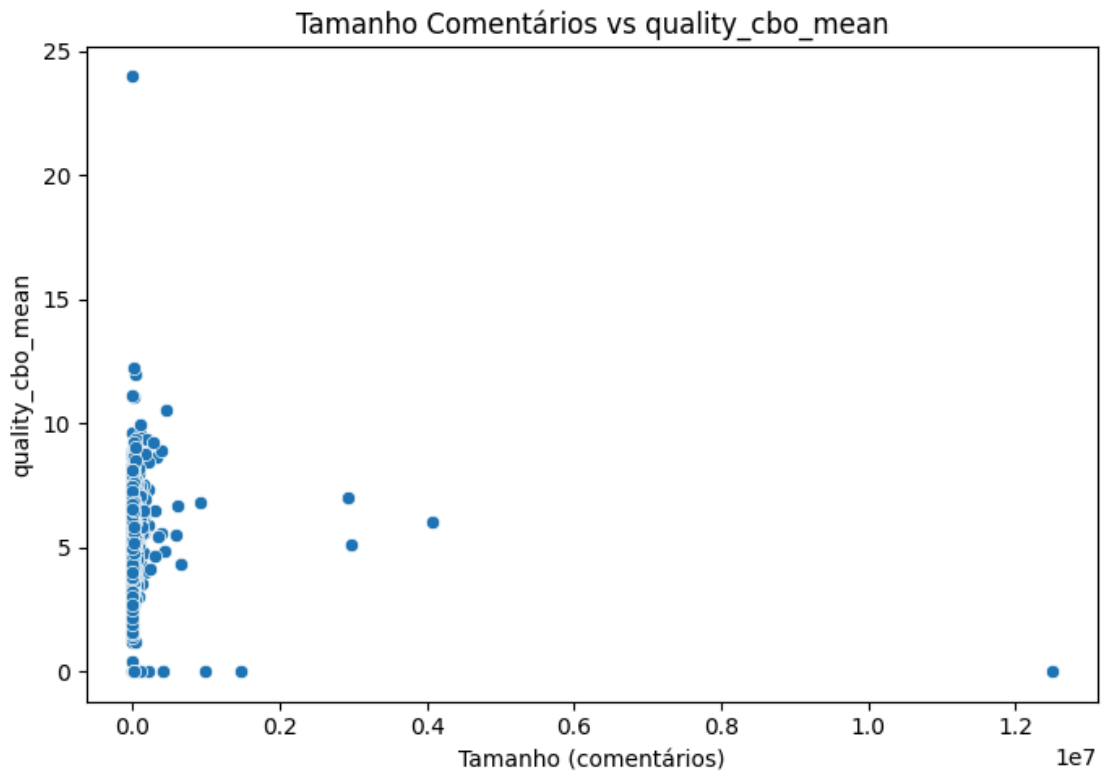
- Figura 12 – Relação entre tamanho (LOC) vs LCOM:



Neste gráfico, analisa-se a relação entre o tamanho do sistema e a coesão das classes. Observa-se que repositórios maiores apresentam maior dispersão nos valores de LCOM.

Esse comportamento sugere que sistemas maiores têm maior probabilidade de apresentar problemas de coesão, possivelmente devido à dificuldade de organização interna em projetos mais complexos.

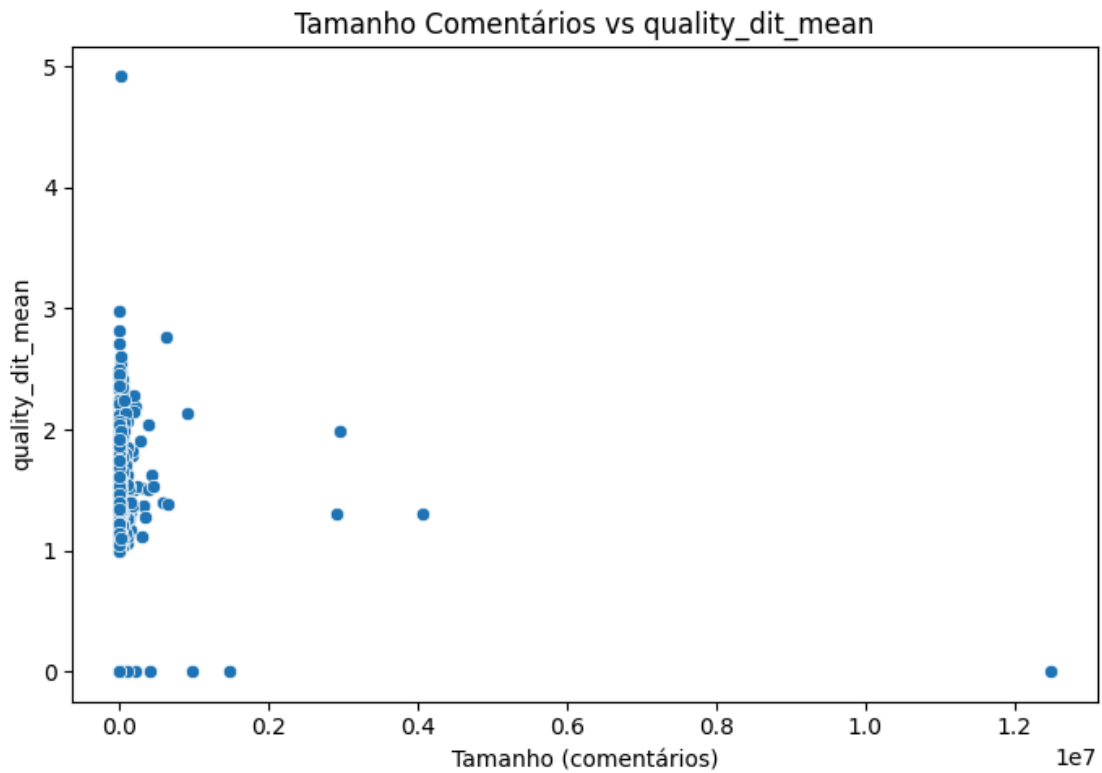
- Figura 13 – Relação entre tamanho (comentários) vs CBO:



O gráfico apresenta a relação entre a quantidade de comentários no código e o acoplamento entre classes (CBO). Observa-se uma grande dispersão dos dados, sem formação de padrão evidente.

A quantidade de comentários não demonstra influência direta sobre o nível de acoplamento do sistema. Repositórios com mais comentários podem apresentar tanto baixos quanto altos valores de CBO, indicando que a documentação do código não está necessariamente relacionada à sua estrutura de dependências.

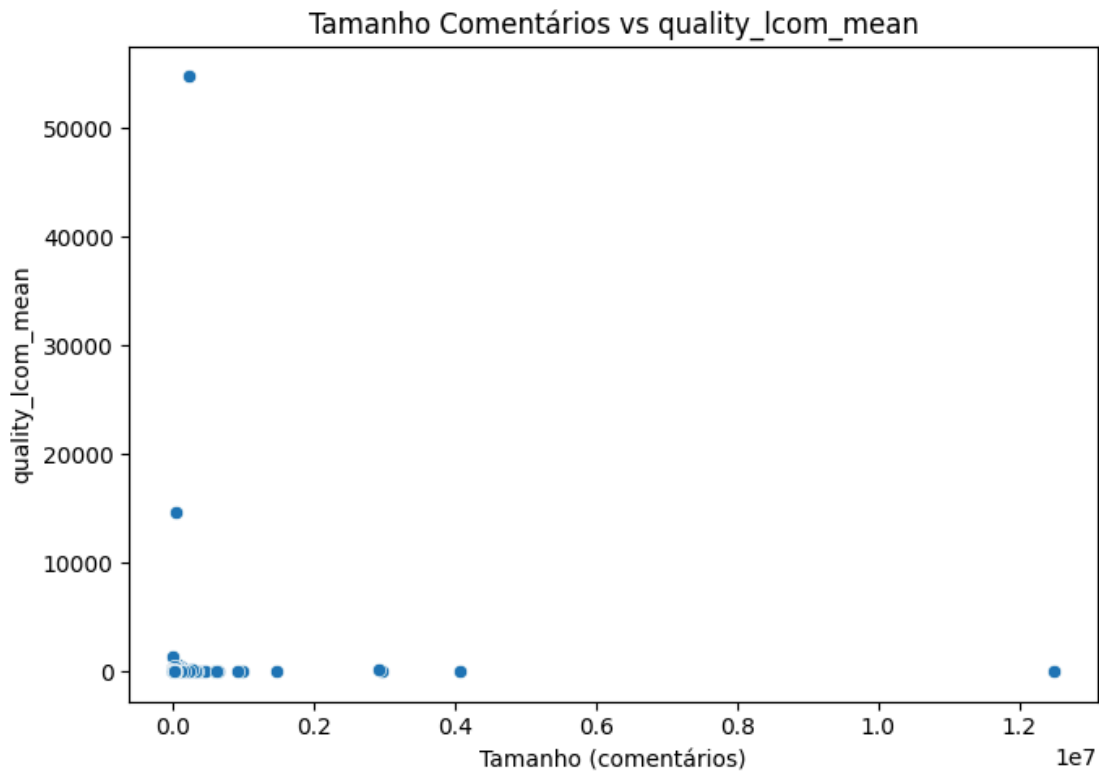
- Figura 14 – Relação entre tamanho (comentários) vs DIT:



Neste gráfico, observa-se a relação entre a quantidade de comentários e a profundidade da árvore de herança (DIT). A distribuição dos dados indica ausência de correlação clara entre as variáveis.

Isso sugere que o volume de comentários no código não está relacionado ao uso de herança, sendo características independentes dentro dos projetos analisados.

- Figura 15 – Relação entre tamanho (comentários) vs LCOM:



O gráfico apresenta a relação entre a quantidade de comentários e a métrica LCOM. Observa-se uma grande dispersão dos pontos, indicando ausência de padrão significativo.

Esse comportamento sugere que a quantidade de comentários não influencia diretamente a coesão das classes. A presença de comentários pode melhorar a legibilidade, mas não necessariamente impacta a organização estrutural do código.

7. Discussão

Com base nos resultados obtidos e nos gráficos apresentados ao longo da Seção 6, é possível identificar algumas tendências relevantes entre as características dos repositórios e as métricas de qualidade analisadas.

Em relação à **RQ01** (popularidade vs qualidade), observa-se que a maioria dos repositórios com baixo número de estrelas apresenta uma grande dispersão nos valores de CBO, indicando variabilidade no nível de acoplamento. Conforme ilustrado no gráfico da página 5, não há uma correlação linear forte entre popularidade e qualidade. Entretanto, repositórios mais populares tendem a concentrar-se em faixas moderadas de acoplamento, sugerindo que projetos amplamente utilizados podem adotar práticas mais

consistentes de organização de código, ainda que não necessariamente apresentem os melhores valores absolutos.

Para a **RQ02** (maturidade vs qualidade), os gráficos indicam uma leve tendência de aumento na complexidade estrutural com o tempo. Conforme observado na página 8, repositórios mais antigos apresentam, em alguns casos, valores mais elevados de DIT, o que pode indicar maior profundidade de herança. Esse comportamento sugere que, com a evolução do sistema, há um crescimento natural da complexidade, possivelmente associado à incorporação de novas funcionalidades e à manutenção contínua.

Na **RQ03** (atividade vs qualidade), os resultados mostram uma relação pouco evidente entre o número de releases e as métricas de qualidade. Como pode ser visto na página 9, a dispersão dos pontos indica ausência de uma correlação clara. Isso sugere que alta atividade não implica necessariamente em melhor qualidade estrutural, podendo refletir apenas maior frequência de mudanças, sem garantia de melhorias arquiteturais.

Por fim, na **RQ04** (tamanho vs qualidade), observa-se uma tendência mais consistente. Conforme apresentado nos gráficos das páginas 11 a 13, repositórios com maior quantidade de linhas de código tendem a apresentar maior variabilidade nas métricas de qualidade, especialmente no CBO e LCOM. Esse comportamento reforça a hipótese de que sistemas maiores são mais difíceis de manter e podem apresentar maior acoplamento e menor coesão.

De forma geral, os resultados indicam que as hipóteses iniciais são parcialmente confirmadas. Algumas relações esperadas, como o impacto do tamanho na qualidade, foram observadas com maior clareza, enquanto outras, como popularidade e atividade, apresentaram comportamento mais disperso e menos conclusivo.

Além disso, é importante destacar que limitações do experimento, como o uso de uma amostra reduzida e problemas técnicos durante a coleta (por exemplo, erros de clonagem e ausência de arquivos Java), podem ter influenciado os resultados obtidos.

Os resultados obtidos estão alinhados com estudos da literatura que indicam que o tamanho do software é um fator relevante na complexidade estrutural. No entanto, a ausência de correlação forte em outras variáveis sugere que a qualidade do software é influenciada por múltiplos fatores, não sendo possível explicá-la por um único atributo isolado.

8. Conclusão

Este trabalho teve como objetivo analisar a qualidade de repositórios Java por meio da utilização de métricas de software, investigando a relação entre atributos externos dos projetos e características internas do código.

A partir da execução do experimento, foi possível validar a viabilidade da coleta automatizada de dados utilizando a ferramenta CK, bem como a consolidação dessas informações em um conjunto estruturado para análise.

Os resultados obtidos indicam que:

- O tamanho do repositório apresenta relação mais evidente com a qualidade, impactando diretamente métricas como acoplamento e coesão;
- A popularidade não garante, por si só, melhor qualidade de código, embora projetos populares tendem a apresentar maior consistência;
- A maturidade pode estar associada ao aumento da complexidade ao longo do tempo;
- A atividade do projeto não demonstrou uma relação clara com a qualidade estrutural.

Dessa forma, conclui-se que a qualidade de software é influenciada por múltiplos fatores e não pode ser explicada por uma única característica isolada. A análise realizada reforça a importância de boas práticas de desenvolvimento e manutenção contínua ao longo do ciclo de vida do software.

Como trabalhos futuros, recomenda-se:

- Ampliar a quantidade de repositórios analisados;
- Aplicar técnicas estatísticas mais avançadas, como análise de correlação;
- Considerar outras métricas de qualidade e fatores externos, como número de contribuidores e frequência de commits.

As questões de pesquisa propostas foram parcialmente respondidas. Observou-se que o tamanho dos repositórios apresenta maior impacto na qualidade do código, enquanto fatores como popularidade e atividade não demonstraram relação significativa.

Por fim, o estudo contribui para a compreensão das relações entre características de projetos open source e a qualidade do código, fornecendo subsídios para futuras pesquisas na área de Engenharia de Software.

9. Referências

ANICHE, Maurício.

CK: Java Code Metrics Tool.

Disponível em: <https://github.com/mauricioaniche/ck>

CHIDAMBER, S. R.; KEMERER, C. F.

A Metrics Suite for Object Oriented Design.

IEEE Transactions on Software Engineering, v. 20, n. 6, p. 476–493, 1994.

PRESSMAN, Roger S.

Engenharia de Software: Uma Abordagem Profissional.

8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian.

Engenharia de Software.

10. ed. São Paulo: Pearson, 2019.

ISO.

ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.

Genebra: International Organization for Standardization, 2011.

FERREIRA, Ivan; BIGONHA, Márcio; BIGONHA, Roberto.

Uma análise empírica de métricas de software em sistemas reais.

Revista de Informática Teórica e Aplicada, 2012.